COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Laboratory Workbook

Execution plans, acceptance checks and graduate problem sets

20 LABS / 40 PROBLEMS / 5 MODULE PROJECTS

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Lab execution and evidence 4

Module 41 laboratory 5

Lab 41.1 — Reading research as claims, assumptions and evidence . . . 6

Lab 41.2 — Conditioning, sparse systems and iterative solvers . . . 8

Lab 41.3 — Constrained optimization, derivatives and optimality conditions . . . 10

Lab 41.4 — Experimental design, uncertainty and reproducible graduate work . . . 11

Module 41 project: numerical research notebook . . . 12

Module 42 laboratory 13

Lab 42.1 — Parallel reductions, scans and work-span analysis . . . 14

Lab 42.2 — Tiling, data reuse and asynchronous pipelines . . . 16

Lab 42.3 — Irregular workloads, wavefront scheduling and load balance . . . 18

Lab 42.4 — Portable performance and integration-quality evaluation . . . 19

Module 42 project: reusable GPU primitive in an engine . . . 20

Module 43 laboratory 21

Lab 43.1 — Monte Carlo light transport and multiple importance sampling . . . 22

Lab 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines . . . 24

Lab 43.3 — Spatiotemporal reuse, reservoirs and denoising . . . 26

Lab 43.4 — Differentiable and neural rendering under engineering constraints . . . 27

Module 43 project: bounded rendering reproduction . . . 28

Module 44 laboratory 29

Lab 44.1 — Large-world scheduling, streaming and critical-path control . . . 30

Lab 44.2 — Deformable simulation, variational models and solver tradeoffs . . . 32

Lab 44.3 — Distributed game state, consistency and overload behavior . . . 34

Lab 44.4 — Formal specifications, replay and reliability-oriented engine design . . . 36

Module 44 project: one engine-system depth experiment . . . 37

Module 45 laboratory **38**

Lab 45.1 — Intermediate representations, SSA and dataflow analysis . . . 39

Lab 45.2 — Alias analysis, vectorization and legal optimization . . . 41

Lab 45.3 — Domain-specific languages, specialization and safe JIT execution . . . 42

Lab 45.4 — Compiler/hardware co-design and graduate artifact review . . . 43

Module 45 project: compiler mini-project plus Gate I . . . 44

Lab execution and evidence

Complete the original lab and every acceptance check, not just the starter function. Use a separate learner directory or repository and preserve the reference implementation unchanged. Each lesson adds two graduate problems; the original three-question quiz stays in the handbook. Attempt problems before reading the key.

For each lab, record the input domain, numerical/semantic contract, predicted result, independent oracle, directed boundary cases, adverse case, resource limits, exact command, raw evidence and explanation. A test pass without a stated contract is not a complete result. Diagnostic reference outputs are supplied in evidence/reference_run; write your own runs elsewhere.

Use templates/LAB_JOURNAL.md for each lesson. File names below are suggested evidence organization, not preexisting learner submissions. Report pending GPU or independent-review work explicitly. A reasonable formative rule is every listed check satisfied and a fresh transfer case explained, with unresolved discrepancies documented and the accepted domain narrowed where appropriate.

Module 41 laboratory

Graduate numerical methods and research practice

Lab 41.1 — Reading research as claims, assumptions and evidence

Target: Extract an implementable argument rather than only a paper summary.

Original assignment and pass condition: Analyze one paper associated with R23, R44 or R55. Pass: a two-page claim map, a reconstructed central algorithm, at least three assumptions and a feasible reproduction target are reviewed by another reader.

Starter: `code/exercises/lesson_41_1.py`

Reference: templates/CLAIM_EVIDENCE_MAP.md; code/run_experiments.py

Execute and record

1. Choose XPBD 2016, ReSTIR 2020, or one method explicitly traced through PBRT references. Save the exact author, year, version and retrieval date; link rather than redistribute the paper.
2. Prepare a two-page claim map with at least three assumptions and one disputed or underspecified implementation choice.
3. Reconstruct the central algorithm on a scalar or discrete example. Annotate input domains, units, invariants and where approximation enters.
4. Define one central target, a competent baseline and a failure criterion. Estimate the implementation and evaluation cost before committing.
5. Ask another reader to identify a claim broader than your evidence. Revise the scope and retain that review comment.
6. Create an environment record and run the package tests as setup evidence, not as research evidence.

Acceptance checks

- The selected publication and version are unambiguous.
- Three assumptions have concrete falsification tests.
- The target controls quality, time boundary and workload.
- An independent reader reviewed the claim map.
- The planned result can be evaluated without inventing unavailable hardware measurements.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/41.1/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R44: abstract, algorithm and evaluation of the 2020 ReSTIR paper; or R55 / S05: XPBD (2016), Sections 3-5. Read the chosen paper in full before finalizing the reproduction boundary. Use S14 for artifact-review vocabulary.

Graduate problems

P1. An abstract says "4x faster" but the figure measures samples per second, excluding preprocessing. Write a narrower claim and one required end-to-end experiment.

P2. Your reproduction misses the reported value by 15%. Design a diagnostic order that separates estimator, scene, precision, parameter and timing-boundary differences.

Lab 41.2 — Conditioning, sparse systems and iterative solvers

Target: Choose a solver from mathematical properties and error requirements.

Original assignment and pass condition: Implement a sparse matrix-vector product and conjugate gradient for a verified SPD system. Pass: derive the relevant assumptions, compare residual and forward error, and evaluate a simple preconditioner over several conditioning levels.

Starter: `code/exercises/lesson_41_2.py`

Reference: code/gradlab/numerical.py; code/tests/test_numerical.py

Execute and record

1. Derive Ae=r and the relative condition-number bound. Reproduce the diagonal counterexample by hand.
2. Implement CSR multiplication in the exercise file, checking malformed structures and empty dimensions.
3. Implement CG/PCG independently and compare a known solution with the dense oracle on small SPD problems.
4. Sweep condition numbers 1, 100, 10000 and 1000000; compare identity and Jacobi preconditioning.
5. Log true residual, forward error, iterations, setup time and solve time. Include max-iteration termination.
6. Add a non-diagonal SPD family to prevent the diagonal preconditioner test from being the whole study.

Acceptance checks

- Known-solution error meets the declared tolerance.
- No result is marked converged solely because a loop ended.
- Standard CG is not advertised as valid for arbitrary matrices.
- Residual and forward error appear separately in raw data.
- Preconditioner setup and application costs are retained.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/41.2/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R54: positive-definite equilibrium systems; S01: Shewchuk, Sections 3, 8, 11-12 and Appendix B3; R42: numerical linear algebra background. Re-derive the two-dimensional update without copying implementation code.

Graduate problems

P1. For A=diag(1,10^-6), compute relative residual and relative forward error for b=(1,10^-6), x=(1,0).

P2. Why might a preconditioner that halves iteration count still lose overall time? Give a break-even formula.

Lab 41.3 — Constrained optimization, derivatives and optimality conditions

Target: Derive and verify an optimization method for an engineering objective.

Original assignment and pass condition: Derive the example, then solve a constrained fitting or IK problem with a numerical method. Pass: derivative checks, feasibility residual, convergence criteria and at least one failure or sensitivity case are reported.

Starter: `code/exercises/lesson_41_3.py`

Reference: code/gradlab/numerical.py: constrained_fit, finite_gradient

Execute and record

1. Derive the equality-constrained example and implement the affine projection without a generic optimizer.
2. Write gradient checks for a quadratic, the two-link arm residual and a selected fitting objective.
3. Implement a damped least-squares or projected constrained fitting/IK experiment beyond the supplied affine oracle.
4. Record feasibility, stationarity or projected-gradient residual, objective history and termination reason.
5. Sweep initialization, finite-difference step size and one scale parameter.
6. Add an unreachable or rank-deficient case and design an explicit fallback rather than mislabeling it convergence.

Acceptance checks

- The Lagrangian sign convention is recorded.
- Both objective quality and constraint feasibility are tested.
- Derivative checks cover a range of step sizes.
- A nonconvex stationary point is not assumed globally optimal.
- The full lab extends beyond the provided closed-form projection.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/41.3/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R42 / S02: Convex Optimization, Sections 4.1, 5.5 and 9.1-9.5; R41: gradients and constrained differentials. Study constraint qualifications separately from convex sufficiency.

Graduate problems

P1. Project t=(1,2,4) onto sum(x)=1. Is the result also a projection onto the probability simplex?

P2. Derive the two columns of the two-link arm Jacobian and identify a singular pose for unit links.

Lab 41.4 — Experimental design, uncertainty and reproducible graduate work

Target: Make a performance or quality claim that survives independent scrutiny.

Original assignment and pass condition: Preregister a small experiment with baseline, primary metric, trial unit and exclusion policy. Pass: raw data and analysis scripts reproduce the tables, and an ablation isolates at least one proposed mechanism.

Starter: `code/exercises/lesson_41_4.py`

Reference: code/gradlab/numerical.py: paired_bootstrap; evidence/reference_run/native_scan_pairs.csv

Execute and record

1. Fill PREREGISTRATION.md before collecting new performance results. Name the primary metric, target population, trial unit, paired design and exclusion policy.
2. Collect a small pilot to estimate noise and runtime; use it to set a feasible sample plan without treating it as the final test.
3. Keep final workloads separate from parameter selection. Execute baseline and candidate in randomized order.
4. Retain raw observations and calculate the chosen effect size with an uncertainty method suitable for the unit.
5. Run one ablation and one parameter sensitivity sweep; report failures and missing cells.
6. Give a reviewer the raw CSV and analysis command; ask them to regenerate the main table without editing values.

Acceptance checks

- The independent unit is justified rather than inferred from row count.
- The main metric and timer boundaries were declared before the final comparison.
- All exclusions and failed configurations are listed.
- Ablation and sensitivity are distinct.
- A reviewer can regenerate the analysis from raw data.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/41.4/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R09: inference, confidence intervals and experimental units; S14: artifact evaluation. Use the package PREREGISTRATION and BENCHMARK_MANIFEST templates before collecting final data.

Graduate problems

P1. One scene produces 10000 frames and another produces 100. How could a frame-weighted mean distort a claim about typical scenes?

P2. Why is the fixed-ratio bootstrap unit test not evidence that the experimental method is statistically sound?

Module 41 project: numerical research notebook

Produce a notebook or script-driven report linking one research claim to a derivation, an independent small oracle and a preregistered evaluation. Implement CSR matrix-vector multiplication and PCG for a verified SPD family. Include at least one non-diagonal family; a diagonal Jacobi example alone is not enough. Compare forward error, true residual, work, setup and solve cost. Add a constrained fitting/IK problem with derivative checks, feasibility and a failure case.

Required artifacts: claim map; equations and units; generated test matrices with known solutions; derivative-step sweep; raw solver histories; preregistration; analysis script; a short reviewer response. A reasonable staged plan is mathematical contract, independent oracle, implementation, then experiments. Correctness 35 points, derivation 25, experimental design 25, reproducibility/limitations 15. Suggested pass: 80/100 with no unexplained numerical mismatch or missing assumption.

Integration checkpoint: explain how solver conditioning or research comparison choices will affect your proposed Gate I track. Do not infer a broad performance conclusion from the included eight-row diagonal diagnostic.

Module 42 laboratory

Advanced parallel and GPU algorithms

Lab 42.1 — Parallel reductions, scans and work-span analysis

Target: Prove the structure of a parallel primitive before tuning it.

Original assignment and pass condition: Implement reduction and either scan or compaction. Pass: derive work/span bounds, test sizes 0, 1 and non-powers of two, and report the numerical/reproducibility contract.

Starter: `code/exercises/lesson_42_1.py`

Reference: code/gradlab/parallel.py; code/native/main.cpp; code/gpu/reduction.cu

Execute and record

1. State identity, associativity and overflow/rounding behavior for the selected operator. Derive work/span bounds.
2. Implement reduction and exclusive scan or stable compaction without using the reference implementation as your submitted work.
3. Test sizes 0,1,3,31,32,33,1023,1024,1025 and a large irregular size. Include all-false and all-true flags.
4. Compare with the serial oracle under the declared numerical contract. Add a floating-point experiment that intentionally demonstrates order dependence.
5. For the GPU route, use multiple dispatches for global hierarchy and validate every boundary lane. Run device error and memory checks where supported.
6. Integrate the primitive into a visibility, active-particle or entity-filtering workload; retain setup and result-consumption costs.

Acceptance checks

- Empty and partial groups behave correctly.
- The proof covers non-power-of-two sizes.
- No ordinary dispatch relies on an unsupported global barrier.
- Integer versus floating-point contracts are explicit.
- GPU completion is marked pending until device execution is actually verified.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/42.1/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

S03: GPU Gems 3, Chapter 39, work-efficient scan and large arrays; R37 / S04: CUDA SIMT programming and synchronization. Treat the historical chapter as algorithmic study, not current device tuning guidance.

Graduate problems

P1. Derive the output length of compaction from flags and exclusive offsets without a second full sum.

P2. Give floating-point values for which regrouping changes the result, then explain why this does not invalidate an exact-integer scan test.

Lab 42.2 — Tiling, data reuse and asynchronous pipelines

Target: Overlap movement and computation without creating races.

Original assignment and pass condition: Implement a tiled matrix multiply or image filter, then one overlap variant where supported. Pass: adversarial dimensions match the reference, synchronization is checked and the end-to-end comparison includes staging overhead.

Starter: `code/exercises/lesson_42_2.py`

Reference: code/gradlab/parallel.py: tiled_gemm, pipeline_trace

Execute and record

1. Derive tile storage and intensity for at least three tile sizes, labeling included and omitted traffic.
2. Implement a tiled multiply or filter and verify against a straightforward reference on rectangular and ragged dimensions.
3. Run pipeline_trace and check that no slot is overwritten before its prior consumer ends. Draw or tabulate the dependency sequence.
4. Implement one overlap variant only on a supported API/device; otherwise submit the model and mark the device experiment pending.
5. Measure one-tile, many-tile and boundary-heavy regimes with staging included.
6. Investigate a losing tile size; explain the loss with resource usage or an explicitly testable hypothesis.

Acceptance checks

- The numerical output matches the declared tolerance.
- Every buffer has an explicit producer-to-consumer and consumer-to-reuse edge.
- Boundary work does not bypass required barriers.
- Modeled timings are labeled separately from measurements.
- The fallback path remains tested.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/42.2/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R37 / S04: shared memory and asynchronous execution; R43: original Roofline model. Read the chosen device API for stage completion and buffer-reuse semantics before attempting an asynchronous implementation.

Graduate problems

P1. Under the stated 4-byte ideal stage model, compute intensity and double-buffer storage for B=16 and B=64.

P2. Four tiles take copy=6 and compute=2 units. Derive ideal serial and two-buffer pipelined completion times.

Lab 42.3 — Irregular workloads, wavefront scheduling and load balance

Target: Design around variable work rather than assuming identical threads.

Original assignment and pass condition: Implement an irregular workload using static assignment and a queue or compaction alternative. Pass: prove or test termination, handle queue capacity safely and measure tail behavior as well as mean throughput.

Starter: `code/exercises/lesson_42_3.py`

Reference: code/gradlab/parallel.py: scheduling_model; code/gradlab/rendering.py: BVH

Execute and record

1. Define the job unit and whether jobs are divisible or can spawn more jobs. Derive lower bounds.
2. Compare static assignment and queue/compaction alternatives on uniform, skewed and burst-arrival distributions.
3. Add a queue-capacity contract and a documented termination protocol to your actual implementation.
4. Create a trace that separates waiting time, service time and final completion.
5. Stress all-idle, full-queue, one-giant-job and jobs-spawning-jobs cases where applicable.
6. Measure scheduling and sorting overhead; integrate into an engine workload rather than timing only an empty queue.

Acceptance checks

- Every accepted job is completed or explicitly canceled, never lost or duplicated.
- Queue capacity cannot be exceeded silently.
- Termination accounts for in-flight producers.
- The analysis states the indivisible-work lower bound.
- No model-only timing is presented as a device result.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/42.3/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R23 / S06: PBRT, Mapping Path Tracing to the GPU; S04: atomics and synchronization. Relate queue publication, consumption and completion accounting to the memory model studied in Module 18.

Graduate problems

P1. Show why an empty queue while one job is executing is insufficient evidence of termination.

P2. A reordering stage costs 0.8 ms and saves 0.5 ms of traversal. When could it still be justified, and what would need measurement?

Lab 42.4 — Portable performance and integration-quality evaluation

Target: Separate algorithmic benefit from one device-specific tuning win.

Original assignment and pass condition: Integrate the selected primitive into the engine and evaluate at least two workload regimes. Pass: application impact is measured, unsupported features fall back safely and any untested device claims are explicitly excluded.

Starter: `code/exercises/lesson_42_4.py`

Reference: code/native/main.cpp --bench; templates/CAPABILITY_MATRIX.md

Execute and record

1. Define a capability matrix with tested, unsupported and untested states. Never collapse them into one pass/fail column.
2. Integrate the chosen primitive into a minimal engine workload with a fallback path.
3. Record compiler, library, driver, device and launch configuration locally; do not borrow another machine's limits.
4. Benchmark small, large and irregular regimes against a competent baseline at the same numerical contract.
5. Compare predicted application benefit with measured total latency and memory footprint.
6. Write a one-page decision that includes a losing case, scope limitations and the maintenance cost of specialization.

Acceptance checks

- The fallback passes the same functional tests.
- Performance claims name the tested target set.
- The chosen baseline is not intentionally weakened.
- Kernel and end-to-end boundaries are both visible.
- No unavailable GPU result is filled with a CPU timing.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/42.4/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R43: Roofline model; S04: capability and launch constraints; R21: the selected graphics/compute API capability contract. Use only measured hardware limits for performance estimates.

Graduate problems

P1. If f=0.05, what is the largest possible application speedup even with an infinitely fast kernel and no overhead?

P2. Distinguish unsupported from untested in a device capability matrix and explain why that distinction matters.

Module 42 project: reusable GPU primitive in an engine

Implement reduction plus scan or compaction, with a declared exact-integer or floating-error contract. Derive work/span, handle empty and partial groups, and use supported inter-dispatch synchronization. Integrate the primitive into an engine workload such as visible-instance compaction, particle filtering or task queue preparation.

Compare a simple independent reference, a competent available library primitive and your implementation. Include small, large and irregular sizes, boundary alignment, dispatch/setup, transfers and application consumption. Record actual target capabilities and fallbacks. A second GPU strengthens portability evidence; without one, state that only the tested device is verified. A CPU model cannot become the missing second device.

Required artifacts: API contract, device capability record, correctness/sanitizer logs where supported, before/after engine trace, raw paired timings, error results and portability matrix. Correctness and synchronization 40 points, integration 25, performance method 20, portability/limitations 15. Suggested pass: 80/100 and no race, numerical-contract violation or fabricated device result. Device evidence pending means the hardware part is not yet passed.

Module 43 laboratory

Advanced rendering and reconstruction

Lab 43.1 — Monte Carlo light transport and multiple importance sampling

Target: Derive estimators with correct densities and measures.

Original assignment and pass condition: Implement a path tracer with explicit light sampling and MIS. Pass: derive the estimator, verify a simple known integral, compare variance at equal cost and test emission and grazing-angle edge cases.

Starter: `code/exercises/lesson_43_1.py`

Reference: code/gradlab/rendering.py; code/run_experiments.py render

Execute and record

1. Derive the estimator and write the probability measure beside every PDF. Include light-selection probability.
2. Verify the known integral and a constant-radiance Lambertian case before rendering a complex scene.
3. Implement or independently extend a path tracer with explicit light sampling and MIS. Compare BSDF-only, light-sampling-only and MIS variants.
4. Test directly visible emission, occluded lights, grazing angles, dark materials and nonuniform sample counts.
5. Measure error against a sufficiently converged linear-radiance reference at equal cost; retain uncertainty across seeds.
6. Audit finite-depth, ray-offset and reconstruction biases. Present images as supporting evidence, not the only evidence.

Acceptance checks

- The chosen densities cover all relevant nonzero contributions.
- Area and solid-angle densities are not mixed.
- MIS weights include sample counts and sum appropriately.
- The reference error is characterized rather than assumed zero.
- A finite-depth image is not labeled an unbiased full-transport solution.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/43.1/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R23: Monte Carlo Integration and Light Transport I; S07: Improving Efficiency, especially MIS; S08: A Better Path Tracer. Work through the PDF measures explicitly before porting any code.

Graduate problems

P1. Convert p_area=0.25 per square meter at distance 2 m and light cosine 0.5 to a solid-angle density.

P2. A path survives roulette with probability 0.25 and has pre-roulette contribution 3. What contribution should a surviving path carry, and what is the expectation?

Lab 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines

Target: Budget traversal, building and shading as one system.

Original assignment and pass condition: Add one ray-traced effect to the engine or build a software hybrid prototype. Pass: dynamic-scene tests include total update cost, missing-information fallbacks and resource synchronization evidence.

Starter: `code/exercises/lesson_43_2.py`

Reference: code/gradlab/rendering.py: BVH, Sphere, path_radiance

Execute and record

1. Implement a BVH or extend the supplied median-split tree; verify every hit against brute force on generated scenes.
2. Create a dynamic scene with topology-preserving movement and a separate topology-changing case requiring rebuild.
3. Compare refit and rebuild with the same rays and geometry snapshots. Record update, trace and total time, plus peak memory.
4. Integrate one hybrid shadow/reflection effect or a software equivalent with an explicit missing-information fallback.
5. For hardware execution, retain validation-layer output and a resource lifetime/synchronization table.
6. Test animated sequences and report one motion regime where the selected policy becomes worse.

Acceptance checks

- Accelerated and brute-force hits agree within the selected numerical tolerance.
- Parallel-ray and inside-bound cases are tested.
- Refit is used only under its supported topology contract.
- The cost table includes updates and synchronization, not just ray traversal.
- Hardware-only claims require hardware evidence.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/43.2/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R23: Primitives and Intersection Acceleration; S09: Vulkan acceleration-structure specification for an optional hardware route. Follow the exact API version used by your project for build/update flags and synchronization.

Graduate problems

P1. Derive the break-even reuse count for two policies with build costs bA,bB and query costs qA,qB when qB<qA.

P2. Why can an animated test invalidate a conclusion obtained from a single screenshot?

Lab 43.3 — Spatiotemporal reuse, reservoirs and denoising

Target: Evaluate variance reduction together with bias, history validity and motion.

Original assignment and pass condition: Reproduce a small weighted-reservoir example or temporal denoiser, then evaluate on motion and disocclusion sequences. Pass: the selection/weight rule is derived, bias limitations are stated and results include sequence-level failure cases.

Starter: `code/exercises/lesson_43_3.py`

Reference: code/gradlab/rendering.py: Reservoir, temporal_filter

Execute and record

1. Derive the weighted-selection invariant and test empirical frequencies, zero weights and invalid inputs.
2. Either derive a bounded reservoir estimator or implement a temporal denoiser with explicit reprojection/validity rules. Do not stop at the selection class.
3. Construct sequences for disocclusion, a camera cut, moving geometry and lighting change. Preserve ground-truth or high-quality reference sequences.
4. Compare no reuse, reuse, and reuse with rejection at equal total budget; label the ablation.
5. Separate estimator bias, reconstruction bias, temporal error and memory/runtime costs.
6. For a ReSTIR reproduction, map every retained state and normalization to the selected paper equations.

Acceptance checks

- The selected sample probability is distinguished from the final estimator weight.
- History rejection tests involve changed surfaces, not only stationary noise.
- All-zero reservoirs have a defined safe outcome.
- Correlation and target changes are explicitly discussed.
- A complete ReSTIR claim is not made from the primitive alone.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/43.3/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R44: Bitterli et al., ReSTIR (2020), estimator and reuse assumptions; R23: sampling theory. The executable Reservoir class is only weighted stream selection, not the complete paper algorithm.

Graduate problems

P1. For fixed candidates with weights (1,2,7), show the first item's final selection probability using the sequential replacement rule.

P2. Why is W/(M*p_hat(Y)) necessary in the simple candidate-resampling estimator, rather than outputting only f(Y)?

Lab 43.4 — Differentiable and neural rendering under engineering constraints

Target: Test learned or gradient-based methods against explicit baselines.

Original assignment and pass condition: Implement a small inverse-rendering optimization or learned reconstruction experiment. Pass: gradient or generalization checks, a non-learned baseline, compute/memory accounting and a distribution-shift failure analysis are included.

Starter: `code/exercises/lesson_43_4.py`

Reference: code/gradlab/rendering.py: inverse_albedo; code/gradlab/numerical.py: finite_gradient

Execute and record

1. Choose the inverse-rendering route or a bounded learned-reconstruction route; document why it fits the available compute.
2. For inverse rendering, derive a loss and analytic reference, check gradients and include an unidentifiable or bound-active case.
3. For learned reconstruction, reserve scenes, establish a non-learned baseline and record the exact training recipe and model artifact.
4. Evaluate a distribution shift or visibility-related failure rather than only training-like frames.
5. Measure preprocessing/training, inference, peak memory and temporal stability under a declared amortization horizon.
6. Report where the method is not trustworthy and when the engine should fall back.

Acceptance checks

- The optimized objective matches the claimed physical or statistical quantity.
- Gradient checks or held-out generalization evidence are present.
- A non-learned baseline is included.
- No training-set result is presented as held-out performance.
- The supplied scalar oracle is not mistaken for a trained network or full inverse renderer.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/43.4/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R47: automatic differentiation and generalization; S10: Mitsuba gradient-based optimization tutorial; R23: the light-transport model you differentiate. Choose an original neural method only for the optional learned route.

Graduate problems

P1. Derive the albedo least-squares optimum and explain what happens when all basis values are zero.

P2. A method needs 600 seconds of preparation and saves 2 milliseconds per frame. How many frames are required to break even on time alone?

Module 43 project: bounded rendering reproduction

Select one result from an original rendering method and reconstruct its estimator or objective. A diffuse MIS renderer, a specific reservoir-reuse variant, a temporal reconstruction method or a smooth inverse-rendering problem can provide the setting, but a primitive alone is not a complete reproduction of a larger paper.

Preserve scene geometry, camera paths, lights, materials, seeds and linear reference images. Measure error against an adequately converged or analytically known reference, including reference uncertainty. Compare at equal total cost and at a declared quality threshold where feasible. Add motion/disocclusion or a justified adverse setting. Separate sampling error from reconstruction bias and include build, shading, history and preprocessing costs appropriate to the claim.

Required artifacts: paper-to-code map, estimator derivation, source scenes, raw linear outputs, error/time analysis, temporal failure evidence, one extension and limitations. Estimator/objective correctness 35 points, baseline/equal-quality comparison 25, temporal/adverse cases 20, artifact/review 20. Suggested pass: 80/100 and no unresolved density-measure or data-leakage error.

Module 44 laboratory

Advanced engine systems and simulation research

Lab 44.1 — Large-world scheduling, streaming and critical-path control

Target: Design a bounded system under changing content and workload.

Original assignment and pass condition: Implement a bounded world-streaming scheduler or large-coordinate experiment. Pass: teleport and memory-pressure tests preserve correctness, peak resources are measured and the critical path is explained with a trace.

Starter: `code/exercises/lesson_44_1.py`

Reference: code/gradlab/systems.py: StreamModel, Request

Execute and record

1. Draw an engine streaming state machine and a frame dependency graph. Declare queue, resident, scratch and staging budgets.
2. Run the supplied bounded model, then add late completions tagged by epoch and an explicit reservation lifecycle.
3. Inject a teleport while work is pending, oversize assets, repeated cancellations, queue saturation and memory pressure.
4. Add a cell-plus-local-coordinate experiment or a rebase trace that includes previous-frame transforms.
5. Record maximum outstanding requests, peak bytes, deadline misses and recovery time; classify placeholders as deliberate degradation.
6. Connect at least one measured/model trace to the critical path and label which time values are analytical rather than device measurements.

Acceptance checks

- Resident memory and transient overlap are separately bounded.
- An obsolete completion cannot publish into the new world epoch.
- Every accepted request ends in completion, rejection or cancellation under the stated progress assumptions.
- The trace includes a teleport and a scarce-memory case.
- Coordinate conventions are explicit at physics/render/network boundaries.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/44.1/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R20 and R26: engine task, world and streaming architecture; R54: numerical representation. The reference is a discrete-event resource model, not asynchronous disk or GPU I/O.

Graduate problems

P1. What is the ideal critical path of A=2, B=3 after A, C=4 after A and D=1 after B and C? Can four workers reduce it?

P2. A resident budget is 100 MB, existing assets use 70 MB and a new 20 MB asset needs 25 MB of overlapping scratch/staging. Does a final-residency-only check safely admit it?

Lab 44.2 — Deformable simulation, variational models and solver tradeoffs

Target: Derive a physical or constraint model before comparing visual results.

Original assignment and pass condition: Implement a small cloth, elastic bar or compliant-constraint model and compare two integration/solver choices. Pass: derive the core equations, check gradients, quantify constraint or energy error and reproduce one bounded result from the assigned research.

Starter: `code/exercises/lesson_44_2.py`

Reference: code/gradlab/systems.py: xpbd_scalar, bar_step, simulate_bar; code/run_experiments.py: bar

Execute and record

1. Derive the scalar spring force, implicit-Euler equation and XPBD update with units and mass weighting.
2. Run the scalar oracle and the bar sensitivity experiment. Check pinned boundaries and zero initial displacement.
3. Compare iterative XPBD with the independent dense solve at identical timestep, material parameters, forcing and simulated duration.
4. Vary timestep and iteration count independently; add a material/stiffness sweep and report residual/error, not only runtime.
5. Reproduce a bounded result or property from S05 with its assumptions mapped to your implementation. Add one honest extension, such as ordering or stiffness sensitivity.
6. Document stability, numerical accuracy and physical-model limitations separately; avoid claiming energy conservation for a dissipative implicit method.

Acceptance checks

- Compliance scales with h squared as specified by the selected formulation.
- Multipliers accumulate within a timestep.
- Reference and candidate solve the same discrete problem.
- The physical duration remains constant across timestep sweeps.
- A paper reproduction is distinguished from a scalar teaching oracle.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/44.2/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

S05 / R55: Macklin, Mueller and Chentanez, XPBD (2016), equations 10-18; R54: variational mechanics and linear systems. Reproduce the selected result before calling an extension research evidence.

Graduate problems

P1. Compute the first and second scalar updates for w=1, C=1 initially, alpha=0.01 and h=0.1 with accumulated multipliers.

P2. If a timestep is halved while physical compliance is held fixed, how does alpha_tilde change? Does this alone prove that trajectories at the two timesteps match?

Lab 44.3 — Distributed game state, consistency and overload behavior

Target: Specify which guarantees survive network delay and partial failure.

Original assignment and pass condition: Model a two-partition entity handoff with injected failure. Pass: a stated safety invariant holds, recovery assumptions are explicit and overload is bounded without unbounded retry amplification.

Starter: `code/exercises/lesson_44_3.py`

Reference: code/gradlab/systems.py: FencedStore

Execute and record

1. State one safety invariant, the commit boundary, ownership-transfer assumptions and what is durably recorded.
2. Run the fenced-store tests and draw the history of a delayed old-owner request.
3. Build a two-partition local process or deterministic event simulation with delay, duplication, timeout and restart events.
4. Test same-ID/same-payload retries, same-ID/conflicting-payload requests, stale epochs and deduplication capacity exhaustion.
5. Add bounded retries and backpressure; quantify offered work versus accepted work during a retry burst.
6. Write a limitations section explaining what a production consensus/storage layer must provide beyond the teaching store.

Acceptance checks

- Stale authority is rejected where the irreversible effect commits.
- Duplicate detection and the effect share an atomic boundary in the stated model.
- Queue and deduplication growth are bounded.
- Safety is tested across a failure sequence rather than only a happy path.
- Durability, scale and network claims do not exceed the tested model.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/44.3/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

S11: Ongaro and Ousterhout, Raft, replicated state-machine assumptions; S17: Consul session and locking semantics. These sources distinguish coordinated authority from a local locking intuition; the supplied store is not a consensus implementation.

Graduate problems

P1. Why is a lease on the sending server insufficient to reject a delayed stale write at an external sink?

P2. A request can be retried three times after its original attempt. What is the maximum attempt amplification for one logical request, and why is that not a sufficient overload policy alone?

Lab 44.4 — Formal specifications, replay and reliability-oriented engine design

Target: Use precise invariants to reduce the space of subtle failures.

Original assignment and pass condition: Model a small engine lifecycle or queue protocol and connect it to implementation tests. Pass: at least one safety and one conditional liveness property are defined, a deliberate violation is found and abstraction limits are documented.

Starter: `code/exercises/lesson_44_4.py`

Reference: code/gradlab/systems.py: lifecycle_check, replay_hash

Execute and record

1. Specify a small lifecycle or queue model with state variables, initial states, transitions and one safety property.
2. State a separate conditional liveness requirement and its fairness/resource assumptions.
3. Run the finite-state safety checker and reproduce the deliberate missing-guard counterexample.
4. Add a realistic feature such as a second request, generation reuse or cancellation acknowledgment; report state-space size and bounds.
5. Map model transitions to implementation tests and create a deterministic event replay with checkpoint state comparisons.
6. Explain which properties were exhaustively checked, tested, argued informally or not evaluated.

Acceptance checks

- The deliberate bug is detected, preventing a vacuous passing result.
- Safety and liveness are not conflated.
- The environment assumptions permit realistic cancellation/release races.
- At least one counterexample is connected to a concrete code test.
- Replay records enough nondeterminism for the claimed execution model.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/44.4/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

S12: Lamport, Safety, Liveness, and Fairness; R50: formal assumptions and bounded verification. The included exhaustive search establishes safety only for its small explicit model.

Graduate problems

P1. Does exhaustive safety exploration of the supplied one-request model establish eventual completion? Explain.

P2. Why should an artifact keep event/state data as well as their hashes?

Module 44 project: one engine-system depth experiment

Choose world streaming, deformable simulation, distributed state or lifecycle reliability as the main depth area. Complete all four lesson labs at their bounded scale, then integrate one area more deeply. Write a numerical argument or state/transition model before comparing implementations.

Streaming evidence must include teleport, late completion, transient memory and a bounded degradation policy. Simulation evidence must separate model, discretization and solve error with timestep/iteration sweeps. Distributed-state evidence must identify the authoritative sink, deduplication/commit boundary and persistence assumptions under fault histories. Reliability evidence must distinguish checked safety from conditional liveness and connect the model to implemented tests.

Required artifacts: contract, reference model, implementation, deliberately failing case, stress trace, parameter sweep and limitations. Model/argument 30 points, implementation correctness 30, failure/overload evaluation 25, reproducibility 15. Suggested pass: 80/100 with no unbounded resource promise or unqualified reliability claim.

Module 45 laboratory

Compilers, shader languages and engine specialization

Lab 45.1 — Intermediate representations, SSA and dataflow analysis

Target: Represent program meaning so transformations can be justified.

Original assignment and pass condition: Extend the tiny compiler with a control-flow IR and one dataflow analysis. Pass: derive transfer functions and a fixed-point stopping condition, test loops and merges, and verify transformed programs against an interpreter.

Starter: `code/exercises/lesson_45_1.py`

Reference: code/gradlab/compiler.py: run_cfg, live_variables

Execute and record

1. Write the accepted IR grammar, type rules, arithmetic contract, terminators and phi edge semantics.
2. Run the CFG examples and add a diamond merge, counted loop and simultaneous-phi swap case.
3. Derive liveness transfer functions and implement a work-list analysis; document phi preprocessing or implement edge-aware uses.
4. Add at least one static verifier check beyond runtime execution, such as predecessor/phi agreement or dominance for ordinary uses.
5. Implement a transformation such as proven-branch simplification or constant propagation, preserving before/after IR.
6. Differential-test interpreter results and explain termination of the analysis separately from termination of the program.

Acceptance checks

- Phi values are selected by the actual predecessor and assigned simultaneously.
- Loops and merges are covered, not only straight-line expressions.
- The dataflow domain, transfer rules and fixed point are stated.
- A failed verifier rejects invalid IR before optimization.
- No full compiler-verification claim is made from a bounded interpreter.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/45.1/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R14 and S13: LLVM control-flow and SSA semantics; use the original tiny compiler from Module 15 as the frontend where available. The supplied CFG interpreter is an executable semantic oracle, not a complete IR verifier.

Graduate problems

P1. For USE[B]={a}, DEF[B]={b}, OUT[B]={b,c}, compute IN[B].

P2. Two loop phis swap x and y on a backedge. Why must their incoming values be selected simultaneously?

Lab 45.2 — Alias analysis, vectorization and legal optimization

Target: Separate transformations that are fast from transformations that preserve meaning.

Original assignment and pass condition: Implement or study a loop transformation with an explicit legality check. Pass: overlapping-memory and boundary cases are tested, compiler flags are recorded and speed claims exclude transformations that violate the required numerical contract.

Starter: `code/exercises/lesson_45_2.py`

Reference: code/gradlab/compiler.py: ranges_overlap, fold; code/native/main.cpp

Execute and record

1. Select a loop or expression transformation and write an explicit arithmetic/aliasing contract.
2. Prove its legality for the accepted subset; list the cases that require a fallback.
3. Add boundary, overlapping-memory and exceptional-value tests appropriate to that contract.
4. Build the C++ reference in Debug and Release, then run a separate sanitizer configuration on a supported compiler.
5. Capture optimizer remarks and generated code for at least two workload sizes; identify guards, tails and spills where present.
6. Measure end-to-end impact with checks outside the timed region and output consumption that prevents dead-code elimination.

Acceptance checks

- Numerical equivalence is required before speed is considered.
- No-alias assumptions match actual caller behavior.
- Remainders and short loops are tested.
- The exact compiler flags and target are recorded.
- Sanitizer timings are not mixed with release performance measurements.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/45.2/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

S13: LLVM language semantics; S15: LLVM loop and SLP vectorizers; R11: lifetime, arithmetic and aliasing contracts. Record actual compiler versions and flags, not an assumed universal optimizer behavior.

Graduate problems

P1. Do the half-open logical ranges [0,8) and [8,16) overlap? What about [0,8) and [7,9)?

P2. Why does a passing randomized test suite not by itself prove an optimization legal for every input?

Lab 45.3 — Domain-specific languages, specialization and safe JIT execution

Target: Trade generality for predictable game-engine computation.

Original assignment and pass condition: Build a typed expression or material DSL with constant folding and a bounded interpreter or JIT route. Pass: invalid graphs fail safely, variant counts are measured and compiled outputs match the reference.

Starter: `code/exercises/lesson_45_3.py`

Reference: code/gradlab/compiler.py: validate_expr, interpret, fold, cache_key, generate_cpp

Execute and record

1. Specify DSL grammar, type/effect rules, node/depth limits and arithmetic behavior.
2. Implement a learner interpreter and compare it with the reference on valid and invalid trees.
3. Add constant folding and verify original-versus-folded results for boundary values and randomized inputs.
4. Generate a bounded trusted C++ corpus or extend your existing compiler; never use eval or shell interpolation to execute graph text.
5. Implement or model a variant cache with complete keys, a size bound and an explicit miss fallback.
6. Document compilation/execution trust boundaries and the protections still required for a real untrusted JIT.

Acceptance checks

- Invalid trees and oversized inputs fail before execution.
- The language has an explicit effects policy.
- Cache keys include target, compiler and semantic flags.
- Cold-start and cache behavior are measured or explicitly modeled.
- Restricted interpretation is not described as an operating-system sandbox.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/45.3/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R14 / S16: LLVM language and JIT tutorials; S13: generated-code semantics; R26: engine material and scripting interfaces. This package emits inspectable C++ but does not execute an untrusted native JIT.

Graduate problems

P1. How many combinations result from eight independent binary options, and what happens after adding two more?

P2. Why can a cache key based only on source text be incorrect after a compiler or target change?

Lab 45.4 — Compiler/hardware co-design and graduate artifact review

Target: Connect a transformation to architecture and application evidence.

Original assignment and pass condition: Complete Gate I and defend one compiler transformation's legality. Pass: an independent reader reruns the central experiment, the claimed extension is precisely identified and raw evidence supports the conclusions.

Starter: `code/exercises/lesson_45_4.py`

Reference: code/run_experiments.py; code/native/main.cpp; templates/INDEPENDENT_REVIEW.md

Execute and record

1. Choose one Gate I depth track and freeze the central claim, baseline and evaluation protocol.
2. Finish the required compiler mini-project even when the research track is elsewhere.
3. Run held-out, adverse, ablation and parameter-sensitivity cases with raw evidence preserved.
4. Regenerate the main table from scripts and connect each row to source, configuration and input identity.
5. Have an independent reviewer reproduce the central result, inspect a failed case and question the derivation or legality argument.
6. Revise the claim to match the evidence and prepare a handoff identifying open questions for Volume 10.

Acceptance checks

- Reproduction and extension are separately named.
- The baseline is competent and comparisons honor the quality contract.
- Ablation, sensitivity and failure evidence exist.
- Independent review is documented, not simulated by a self-check.
- The compiler requirement and any pending device validation remain visible.

Evidence to submit

Store derivation, source, test output, raw observations and an interpretation in `learner_evidence/45.4/`. Include at least one failed prediction or adverse case and explain it. Record timing/model boundaries and any untested capabilities. Use the common lab journal and the focused reading below.

R14, R48 and R49: compiler/architecture interfaces and co-design; S14: artifact review principles. Gate I uses this curriculum's own rubric and does not award an ACM badge or an accredited degree.

Graduate problems

P1. Compute the application speedup in the worked example and explain why the 2x kernel number is insufficient.

P2. A reviewer can run the code but cannot regenerate the main table because scene files are missing. Has Gate I passed?

Module 45 project: compiler mini-project plus Gate I

The compiler requirement remains even when Gate I specializes elsewhere. Extend the prior tiny compiler or the supplied restricted reference with a verified IR subset and a dataflow/optimization pass. Cover loops and merges, state phi/alias/arithmetic semantics, and compare transformed execution against an interpreter. Inspect actual generated code and measure a bounded integration workload, not only expression evaluation in isolation.

Required artifacts: accepted language/IR contract, verifier checks, transfer functions or legality proof, original/transformed IR, directed and randomized tests, compiler flags, generated-code inspection and end-to-end measurements. Semantics 40 points, analysis/verifier 25, generated-code/performance evaluation 20, artifact/limitations 15. Suggested pass: 80/100 with no semantic mismatch. An existing constant-folding smoke test alone does not satisfy this project.

Complete Gate I using CAPSTONE_SPEC.pdf. The compiler mini-project and Gate I can overlap when compilation is the chosen depth area; otherwise they are distinct requirements within the volume's planning estimate. Independent review is required for Gate I, not silently replaced by the reference test suite.